# DAT171 Object oriented programming in Python

- Lecturers:
  - Thomas Svedberg, tel. 772 1522, thomas.svedberg@chalmers.se
  - Mikael Öhman, tel. 772 3191, mikael.ohman@chalmers.se
- Assistant:
  - Sebastian Everett Eriksson sebastian.vasser@gmail.com
- Find Thomas Material & Computational Mechanics (3rd floor M-building)
- Find Mikael at Physics / C3SE office (5Th floor Origo building)

Course literature:

- C. Horstmann: Python for everyone (*optional*, permitted on exam)
- Python.org tutorial
- Reference manuals (available on exam)
- Lecture notes (available on exam)

# Prerequisites

Good knowledge of basic programming structures such as variables, conditions, loops, and functions in any programming language.

| Matlab | Python |
| --- | --- |
| `function [r] = norm(x)` | `def norm(x):` |
| `s = 0;` | `    s = 0` |
| `for i = 1:length(x)` | `    for i in range(len(x)):` |
| `   s = s + x(i)^2;` | `        s += x[i]**2` |
| `end` | |
| `r = sqrt(s);` | `    return sqrt(s)` |

# Course evaluation

- Course evaluated at 2 occurrences
- Course survey after exam
- Course board representatives (please stand up)

| | |
|---|---|
| Hugo Brokelind | hugobr@student.chalmers.se |
| Jayesh Rajendrabhai Gharte | gharte@student.chalmers.se |
| Bowen Jiang | bowen9314@163.com |
| Gabriel Skoghäll | skoghall@student.chalmers.se |

# Content

- Basic building blocks of a Python program (variables, conditional statements, loops, libraries, functions, errors)
- Data structures (lists, sets, dictionaries, tuples)
- Object Oriented programming (classes, objects, inheritance, polymorphism, abstract classes)
- NumPy (Matrices, vectors, linear algebra)
- SciPy (Package for numerical computations)
- Matplotlib (Plotting)
- Qt for creating graphical user interfaces and *interactive* programs
- Interactive Python (IPython)

# Learning outcome

After successfully passing the course, the student should be able to independently write *object oriented* software using *Python 3*.

Furthermore, the student should be able to read (and understand!) reference literature for the Python programming language as well as being able to use the SciPy/NumPy package for numerical computations and Qt for creating graphical user interfaces.

# Examination

The exam will take place in computer rooms. The re-exams are taken in June and in August.

- Book is allowed...
- but there will also be manuals and lecture notes available on the computers
- 5 hours examination
- 25 points, 4-5 questions
  - 10 points for passing grade
  - 15 points for grade 4
  - 20 points for grade 5
- Will get back to you later when rooms are booked.
- The exam will not contain anything on the GUI (5 hours would not be sufficient)

# Computer Assignments

3 *compulsory* assignments worth 1 hp each which represents 26 hours of work per student. Computer assignments should be done in groups of two (or alone, only if OK:ed with teacher!).

Do not cheat! Ask each other questions, but don't share sections of code etc. directly, it will not help you in the long run. *WE'RE REQUIRED TO REPORT CHEATING*

It is OK to find inspiration from the internet, but for your own sake, please understand what the code you are mimicking is doing!

# Assignment 1: Creating a graph of adjacent points and finding the cheapest route

From a file containing coordinates, the goal is to construct a adjacency graph of the coordinates and find the cheapest path between the given start and end point.

- Getting familiar with Python
- Reading files
- Parsing text
- Using libraries
- Using NumPy for numerics
- Using Matplotlib for plotting
- Using more advanced data structures (KDTree and sparse graph library from SciPy)
- Writing *well structured* and documented Python code

# Assignment 2: Playing card library

The task of this assignment is to create a library for making card games.

- Creating a library
- Creating classes
- Operator overloading

# Assignment 3: Simple interactive application

Create a simple interactive graphical application from scratch.

- Create an interactive application
- Creating a GUI with Qt
- Creating classes with inheritance

# Software

Software will be available on the Windows machines in ES61/62, E-Studion and MT0 (we will primarily be in the E-building!).

- Python 3.5 (or later)
- PyQt
- NumPy, SciPy, Matplotlib
- Recommended Integrated Programming Environment (IDE): PyCharm (community edition) https://www.jetbrains.com/pycharm/download/
- Laptops:
  - Anaconda http://continuum.io/downloads
  - Linux: Use your regular package manager (or Anaconda)
  - Mac, Windows: Use Anaconda

# Consultation

- E-mail is preferred before the Ping-Pong PM system
- 3 computer sessions each week (you're expected to work on the assignments on your own in-between sessions!)
- Thomas office on 3rd floor in the M-building (Material & Computational Mechanics / Material & Beräkningsmekanik).
- Mikaels office at Physics / C3SE office (5Th floor Origo building)

# Schedule

- L = Lectures Mon: HA3, Thu: HA2
- CS = Computer lab session Mon, Thu: ES61 & ES62, Fri: E-studion & MT0

| Time | Mon | Tue | Wed | Thu | Fri |
|---|---|---|---|---|---|
| 8-10 | | | | L | |
| 10-12 | | | | CS | |
| 13-15 | L | | | | CS |
| 15-17 | CS | | | | |

- Extra lecture on Friday 13-15 (Room HA3) first week!

# Lecture overview

| W | Content |
|---|---|
| 1 | Introduction, lists and loops, functions, libraries |
| 2 | CA1, files and strings, NumPy and Matplotlib |
| 3 | SciPy, common data structures, writing libraries |
| 4 | CA2, Classes and object oriented design |
| 5 | More on classes, exceptions |
| 6 | CA3, interactive software, GUI: PyQt |
| 7 | More GUI. leftovers, reserve (if needed) |
| 8 | Repetition and questions, review of example exam |
| 9 | Exam, final deadline on assignment corrections |

# Computer sessions

- W1: Introduction and basic Python
  - Bring laptops if you need help installing the software
  - Practice problems (not mandatory)
- W2 ->: Work on Computer Assignments
- We don't expect you to finish the assignments in only the scheduled time!

# Diagnostic questionnaire

You . . .

1. know another Object Oriented programming language already
2. have tried another OO-programming language already
3. thought the Matlab course was easy
4. can manage Matlab decently
5. didn't even satisfy the course prerequisites

# About Python

- Garbage collected (like Matlab, Java, unlike C/C++)
- Dynamically typed (like Matlab, unlike Java, C/C++)
- Compiles to byte-code (like Java, Matlab in some cases, C/C++ completely compiled)
- Object Oriented programming (like Java, C++, unlike Matlab, C)
- Has concept of references (like Java, C/C++, unlike Matlab)
- Interactive interpreter (like Matlab, unlike Java, C/C++)
- Compact syntax (like Matlab, unlike Java)
- Used for scripting support in **many** software applications

# Why Python ?

- Simple syntax
- Easy to get started with as a programming language, as well as on all platforms
- Similar concepts as other popular Object Oriented languages (Java, C++, C#)
- Common scripting language used in many applications, e.g.

```
[C library] <-> [Python wrapper] <-> [GUI code]
[C++ program] <-> [Evaluate user scripts]
```

# Python 3 vs Python 2

- We'll be sticking to Python 3 (3.5 / 3.6 / 3.7)
- A few incompatible syntax changes (compared to Python 2)
- A few notable differences:
  - Printing: `print x` $\rightsquigarrow$ `print(x)`
  - Integer division `a / b` $\rightsquigarrow$ `a // b`
  - Proper Unicode
  - Dictionaries: `x.has_key(y)` $\rightsquigarrow$ `y in x`
  - Renamed functions: `y.iteritems()` $\rightsquigarrow$ `y.items()`
- Python 2.6-2.7 very much widespread, but Python 3.x is the future!

# Running Python code

- Running a single program, once:
  - `$ python myscript.py  inputarguments`
  - where `inputarguments` typically are file names or options used with command line interfaces. (on Windows, this is what happens when you choose "open with" python.exe on a Python script)
- First time running a script will generate a `pyc` file. This is compiled byte-code that is cached (safe to remove).
- Running a single program Matlab-style
  - `$ ipython`
  - `> In [1]: %run myscript.py`
- Running only
  - `$ python`
  - starts an interactive interpreter.
- More convenient to run scripts through an IDE when developing.

# Using an IDE (PyCharm)

- An Integrated Development Environment (IDE) is a pimped up text editor that integrates the tool-chain.
- Running a script
  1. Create a small script `foobar.py`
  2. Run → Run... and select your script.
- Python console at the bottom for convenience
- PyCharm can read IPython notebooks (you might need to start the notebook server on your computer)
- Learn the keyboard shortcuts!
  - Ctrl + Mouse click for code navigation
  - Ctrl + Shift + arrow to move lines around
  - Many other useful things
- You are free to use any IDE you want, but PyCharm is the best tool you will have access to during the exam.

# Short note on VCS

- Version control systems (VCS) is a key component when developing and collaborating on larger projects
  - Subversion (a.k.a SVN) - Common centralised VCS
  - GIT - Distributed version control system. <www.github.com> is a popular hosting service.
  - Any decent IDE will integrate support for VCS.
  - Dropbox is NOT a VCS.
- Not really critical in this course
  - Your programs will mostly be short, and a learning experience
  - If you are interested, feel free to ask about it more during the computer session

# IPython

- IPython has convenient macros outside the Python language
  - Similar to the Matlab console
  - Syntax highlighting warnings/errors (inside the console!)
    - Magic functions

```
%timeit x = sqrt(37)
10000000 loops, best of 3: 52.4 ns per loop
```

- Jupyter notebooks; "*The Jupyter Notebook is an open-source web application that allows you to create and share documents that contain live code, equations, visualizations and narrative text. Uses include: data cleaning and transformation, numerical simulation, statistical modeling, data visualization, machine learning, and much more.*"

  Examples: (click to open in browser)

  SymPy

  Plotly

# IPython

- Use as a calculator
  - Using previous outputs:

```
In [1]: 2**3 + 75*2 - 45/4
Out[1]: 146.75
In [2]: 31536000 / Out[1]
Out[2]: 214896.0817717206
```

    Like 'ans' in Matlab but better!

- Run a script similar to Matlab console

```
In [1]: %run myscript.py
In [2]: whos
Variable   Type    Data/Info
-----------------------------
x          int     3
```

# . . . IPython

- Starting a notebook session
  - `$ ipython notebook` (older versions)
  - `$ jupyter notebook` (newer versions)
  - Windows: Run IPython notebook from start menu

  Use web browser or PyCharm to work with documents (browser opens automatically as you open IPython notebook)